Assignment 2 CO2

Name: Riyaz Kamal .S.D

Register Number: 192572049

Course: DSA0117A

GitHub submission link

https://github.com/Riyazkamal78/DSA0117A_assignments/tree/main/Assignment2_CO2_192572049

1) Call by Value Reference and Pointer

Parameter behaviour and interface decision

A value parameter is a separate long long object initialized from the caller. A reference is an alias for the caller's object; the language does not require a separate reference object or stack slot. A pointer parameter is a copied address whose dereference designates the caller's object. Consequently only the reference and pointer versions modify the stored fee. The pointer version checks for null. A const reference is appropriate for read only access to a large Student object whose lifetime covers the call. For a small scalar, passing by value is usually simpler.

Prediction and explanation

In change(), a is a local copy, b aliases the caller's b, and c points to the caller's c. The first line is 1 12 13. bigger(a,c) returns c by reference because 13 is larger than 1; assignment changes c to 100. show(1) uses y=5 and z=10, show(1,2) uses only z=10, and show(1,2,3) uses no defaults. Their sums are 16, 13 and 6, each followed by a space. applyByValue changes its copy, leaving hostelFee at 45000; the completed reference version leaves 40500. The observed outputs match the predictions.

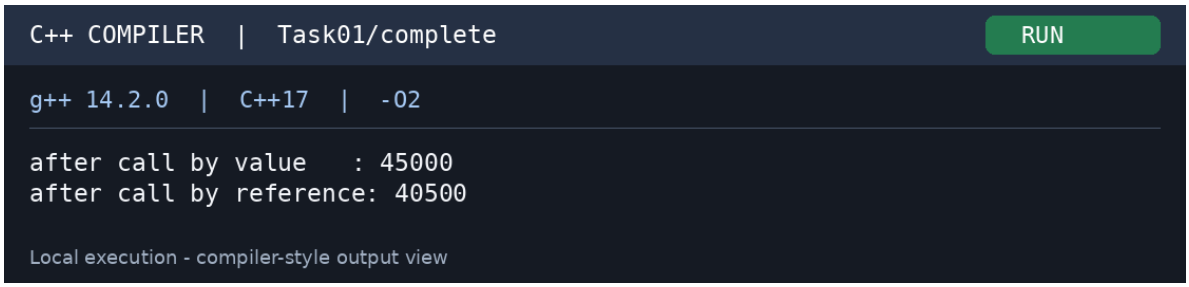
Alternative and complexity

I choose a function that returns the reduced fee: long long reduced(long long fee, int p) { return fee - fee*p/100; }. The caller explicitly assigns fee = reduced(fee,p), making updates visible and allowing calculation before committing a change. Successive 10% and 15% concessions produce 34425; they are not a 25% combined reduction. The rule for combining concessions must be agreed with the hostel office. These examples use nonnegative fees and percentages from 0 to 100, with values small enough to avoid overflow. Integer division truncates the discount.

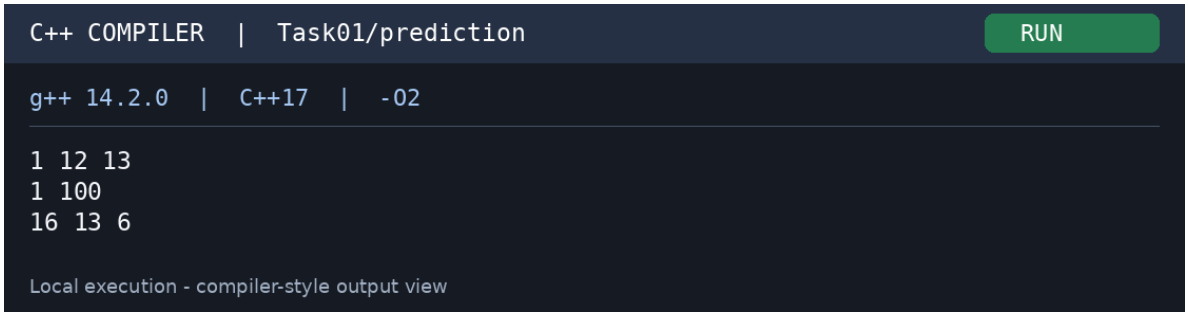
Cost comparison

For long long, all interfaces take O(1) time and O(1) extra space: value copies sizeof(long long) bytes, reference and const reference copy no fee payload, and a pointer copies sizeof(long long*) bytes. For a fixed size Student with S bytes, value copying costs O(S) time and storage, while references use O(1) additional storage and no object copy. A dynamically owned Student may require a deep copy proportional to its owned data, depending on its copy constructor. Returning a large value may benefit from copy elision or moves.

1) Execution evidence







1) Program source

complete.cpp

```
#include <iostream>
void applyByValue(long long fee,int percent){fee-=fee*percent/100;}
void applyConcession(long long& fee,int percent){fee-=fee*percent/100;}
int main(){long long hostelFee=45000;applyByValue(hostelFee,10);
 std::cout<<"after call by value   : "<<hostelFee<<'\n';applyConcession(hostelFee,10);
 std::cout<<"after call by reference: "<<hostelFee<<'\n';}
```

concessions.cpp

```
#include <iostream>
void byValue(long long fee,int p){fee-=fee*p/100;}
void byReference(long long& fee,int p){fee-=fee*p/100;}
void byPointer(long long* fee,int p){if(fee) *fee-=*fee*p/100;}
long long reduced(long long fee,int p){return fee-fee*p/100;}
int main(){long long a=45000,b=a,c=a;
 byValue(a,10);byReference(b,10);byPointer(&c,10);
 std::cout<<"Value: "<<a<<"\nReference: "<<b<<"\nPointer: "<<c<<'\n';
 std::cout<<"Merit then sports: "<<reduced(reduced(45000,10),15)<<'\n';
 std::cout<<"Single parent: "<<reduced(45000,25)<<'\n';}
```

prediction.cpp

```
#include <iostream>
void change(int a,int& b,int* c){a+=10;b+=10;*c+=10;}
int& bigger(int& p,int& q){return p>q?p:q;}
void show(int x,int y=5,int z=10){std::cout<<x+y+z<<' ';}
int main(){int a=1,b=2,c=3;change(a,b,&c);
 std::cout<<a<<' '<<b<<' '<<c<<'\n';bigger(a,c)=100;
 std::cout<<a<<' '<<c<<'\n';show(1);show(1,2);show(1,2,3);std::cout<<'\n';}
```

2) Return by Reference and Its Dangers

Working design and result

highest() returns an alias to an element of the caller's live array. The scan uses the first maximum if marks tie and rejects an empty range. Adding 2 changes 91 to 93; highest(a,3)=95 then changes that same element to 95. highestConst() permits reading but prevents assignment through its const reference. highestCopy() returns an independent int, so changing the copy to zero leaves the stored mark at 95. The caller must supply a valid array containing at least n elements.

Dangling reference evidence

bad() returns a reference to local. That object's lifetime ends on return, so the reference dangles. The compiler warning is recorded in dangling_warning.compiler.txt and in the output image. The warning example is compiled and its harmless main is executed, but bad() is deliberately never called or dereferenced. Undefined behaviour has no reliable expected output, even when a run appears to work.

Growing vector and alternative interfaces

An index can be used as marks[highestIndex(marks.data(), marks.size())] += 2. A pointer can be used as *highestPointer(marks.data(), marks.size()) += 2. A vector reallocation invalidates pointers and references to its elements. An index survives reallocation when the same logical element remains at that index, but insertion, erasure, sorting, or clearing can change its meaning or invalidate its bounds. I would store an index and reacquire access after checking bounds, or recompute the maximum after a mutation. Reserving capacity postpones reallocation but does not guarantee permanent validity.

Complexity and safety

Reference, index and pointer versions all scan n marks in O(n) time with O(1) extra space. Returning the maximum by copy has the same scan cost. A pointer can represent absence with nullptr; a reference requires a valid referent. Neither a pointer nor a reference extends the element's lifetime. An index avoids retaining an address but requires bounds and identity checks after changes.

2) Execution evidence

C++ COMPILER | Task02/dangling_warning RUN

g++ 14.2.0 | C++17 | -O2

```
Task02/dangling_warning.cpp: In function ‘int& bad()’:
Task02/dangling_warning.cpp:1:32: warning: reference to local variable
‘local’ returned [-Wreturn-local-addr]
    1 | int& bad(){int local=42;return local;}
      |                                ^~~~~
Task02/dangling_warning.cpp:1:16: note: declared here
    1 | int& bad(){int local=42;return local;}
      |                ^~~~~

Warning example compiled; main exited 0.
bad() was not called.
```

Local execution - compiler-style output view

C++ COMPILER | Task02/marks RUN

g++ 14.2.0 | C++17 | -O2

```
After moderation: 93
After assignment: 95
Copy: 0, stored: 95
Index: 1, pointer value: 95
```

Local execution - compiler-style output view

2) Program source

dangling_warning.cpp

```
int& bad(){int local=42;return local;}
int main(){/* Deliberately do not call bad: a returned local reference dangles. */}
```

marks.cpp

```
#include <iostream>
#include <stdexcept>
int highestIndex(const int* a,int n){if(n<=0)throw std::invalid_argument("empty");
 int best=0;for(int i=1;i<n;++i)if(a[i]>a[best])best=i;return best;}
int& highest(int* a,int n){return a[highestIndex(a,n)];}
const int& highestConst(const int* a,int n){return a[highestIndex(a,n)];}
int highestCopy(const int* a,int n){return highestConst(a,n);}
int* highestPointer(int* a,int n){return &highest(a,n);}
int main(){int a[]={72,91,84};highest(a,3)+=2;
 std::cout<<"After moderation: "<<a[1]<<'\n';highest(a,3)=95;
 std::cout<<"After assignment: "<<a[1]<<'\n';int copy=highestCopy(a,3);copy=0;
 std::cout<<"Copy: "<<copy<<", stored: "<<highestConst(a,3)<<'\n';
 std::cout<<"Index: "<<highestIndex(a,3)<<", pointer value: "<<*highestPointer(a,3)<<'\n';}
```

3) Default Arguments and Function Overloading

Diagnosis and redesign

charge(2.0) is ambiguous because charge(double) and charge(double,bool=false,bool=false) are both viable with the same exact first argument match. charge(2.0,true) is also ambiguous between the two and three parameter declarations. Defaults affect viability, not overload ranking, and using fewer defaults does not make an overload better. The redesigned overloads accept int grams, double kilograms, or const Parcel&. Only the numeric overloads have trailing express and insured defaults. The illustrative tariff is 50 per kilogram, 100 extra for express and 25 for insurance.

Six predictions and observed selections

charge(2) selects int grams by exact match and costs 0.10. charge(2.0f) selects double kilograms by floating point promotion and costs 100.00. charge(2000,true) selects int grams and costs 200.00. charge(2.0,true,true) selects double kilograms and costs 225.00. charge(Parcel{3.0,false,true}) selects the Parcel overload and costs 175.00. charge(1.5) selects double kilograms and costs 75.00. These predictions were saved before execution; all selections and charges matched.

Resolution rules and defaults

Among these ordinary non template overloads, an exact standard conversion sequence ranks above a promotion, which ranks above an ordinary conversion. Other C++ rules resolve reference binding and template cases; counting conversions alone is not a complete general algorithm. float to double is a promotion, while float to int is a conversion. Default arguments must form a trailing suffix in this ordinary declaration so positional arguments cannot skip an earlier parameter. charge(2) means two grams, a potential usability trap that should be documented or removed with explicit unit types.

Alternative design and cost

A single charge(const Parcel&) interface with fields for weight and options avoids positional boolean arguments. A new option becomes a field with a default. A constrained template can accept several weight types, but still needs an explicit unit policy. Overload resolution happens at compile time with no run time dispatch cost here. Calculation takes O(1) time and space. Numeric arguments are fixed size copies; const Parcel& copies no payload. Passing this fixed Parcel by value copies sizeof(Parcel) bytes including padding, still O(1).

3) Execution evidence

C++ COMPILER | Task03/ambiguous RUN

g++ 14.2.0 | C++17 | -O2

```
Task03/ambiguous.cpp: In function ‘int main()’:
Task03/ambiguous.cpp:4:18: error: call of overloaded ‘charge(double)’ is
ambiguous
    4 | int main(){charge(2.0);}
      |            ~~~~~~^~~~~
Task03/ambiguous.cpp:1:6: note: candidate: ‘void charge(double)’
    1 | void charge(double){}
      |      ^~~~~~
Task03/ambiguous.cpp:3:6: note: candidate: ‘void charge(double, bool,
bool)’
    3 | void charge(double,bool=false,bool=false){}
      |      ^~~~~~
```

Local execution - compiler-style output view

C++ COMPILER | Task03/courier RUN

g++ 14.2.0 | C++17 | -O2

```
int grams: 0.10
double kg: 100.00
int grams: 200.00
double kg: 225.00
Parcel: 175.00
double kg: 75.00
```

Local execution - compiler-style output view

3) Program source

ambiguous.cpp

```
void charge(double){}
void charge(double,bool){}
void charge(double,bool=false,bool=false){}
int main(){charge(2.0);}
```

courier.cpp

```
#include <iostream>
#include <iomanip>
struct Parcel{double kg;bool express=false,insured=false;};
double cost(double w,bool e,bool i){return w*50+(e?100:0)+(i?25:0);}
double charge(int grams,bool e=false,bool i=false){std::cout<<"int grams: ";return
cost(grams/1000.0,e,i);}
double charge(double kg,bool e=false,bool i=false){std::cout<<"double kg: ";return cost(kg,e,i);}
double charge(const Parcel& p){std::cout<<"Parcel: ";return cost(p.kg,p.express,p.insured);}
int main(){std::cout<<std::fixed<<std::setprecision(2);
 std::cout<<charge(2)<<'\n';std::cout<<charge(2.0f)<<'\n';
 std::cout<<charge(2000,true)<<'\n';std::cout<<charge(2.0,true,true)<<'\n';
 std::cout<<charge(Parcel{3.0,false,true})<<'\n';std::cout<<charge(1.5)<<'\n';}
```

4) Savings Account with a Transaction Array

Class specification and execution

SavingsAccount keeps the account number, balance, ten signed transaction values and count private. Deposit is defined inside the class; withdraw and miniStatement are defined outside with SavingsAccount::. Only positive deposits and affordable positive withdrawals are recorded. The example uses integer monetary units, avoiding binary floating point rounding. A production account would also check arithmetic overflow and use transaction identifiers and dates.

Eleventh transaction and invariant

The invariant is 0 <= count <= K, and transactions[0..count) are chronological. On a full array, record shifts entries 1 through K-1 left, reduces count, and appends the new value. Deposits 100 through 1100 discard 100 and retain 200 through 1100; balance is 7600. Withdrawing 200 appends -200, discards 200, and leaves balance 7400. The unaffordable withdrawal is rejected. Private storage prevents callers bypassing these bounds and balance checks.

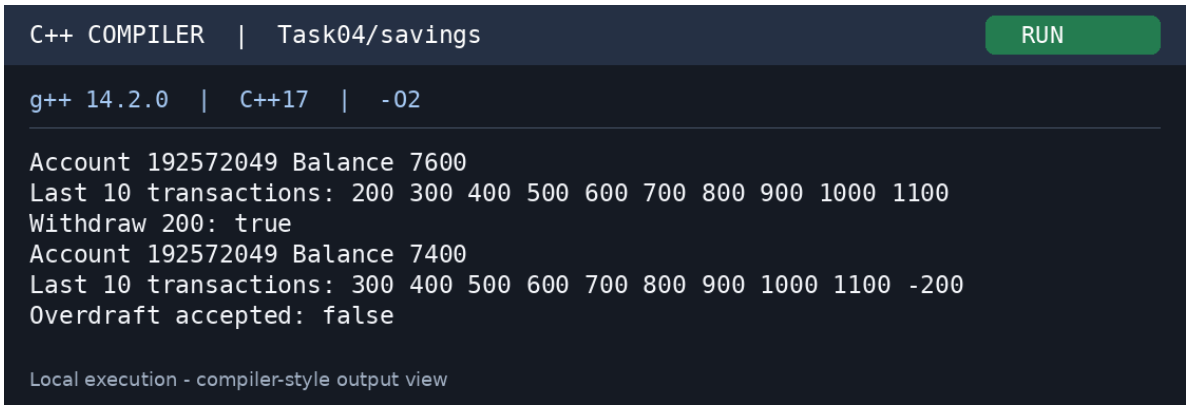
Alternative concept and design decision

A deque can append with push_back(value) and, if size exceeds K, pop_front(). Both end operations are O(1), with O(K) retained data and implementation dependent allocation overhead. A circular array keeps start and count: on a full buffer, write a[start]=value and set start=(start+1)%K; display a[(start+i)%K]. It has O(1) insertion, O(K) display and O(K) storage. Shifting costs O(K) per full insertion and O(K) storage; at a fixed ten it is bounded constant work, but varies linearly with a configurable capacity.

Growth to fifty

Changing K from 10 to 50 is sufficient for the existing capacity based array code, but increases full insertion copying from nine to forty nine shifts. I choose a capacity parameterized circular buffer for growth: the same indexing logic works at both capacities, preserves chronological display and avoids shifting. It adds one start index and a slightly more complex traversal. Q7 measures this redesign directly.

4) Execution evidence



4) Program source

savings.cpp

```
#include <array>
#include <iostream>
#include <string>
class SavingsAccount{
 static constexpr int K=10;
 std::string number;long long balance;std::array<long long,K> transactions{};int count=0;
 void record(long long v){if(count==K){for(int i=1;i<K;++i)transactions[i-1]=transactions[i];--
count;}transactions[count++]=v;}
public:
 SavingsAccount(std::string n,long long b):number(n),balance(b){}
 void deposit(long long amount){if(amount>0){balance+=amount;record(amount);}}
 bool withdraw(long long amount);
 void miniStatement()const;
};
bool SavingsAccount::withdraw(long long amount){if(amount<=0||amount>balance)return
false;balance-=amount;record(-amount);return true;}
void SavingsAccount::miniStatement()const{std::cout<<"Account "<<number<<" Balance
"<<balance<<"\nLast "<<count<<" transactions:";
 for(int i=0;i<count;++i){std::cout<<' '<<transactions[i];}std::cout<<'\n';}
int main(){SavingsAccount a("192572049",1000);for(int i=1;i<=11;++i)a.deposit(i*100);
 a.miniStatement();std::cout<<std::boolalpha<<"Withdraw 200:
"<<a.withdraw(200)<<'\n';a.miniStatement();
 std::cout<<"Overdraft accepted: "<<a.withdraw(99999)<<'\n';}
```

5) Static Members and Arrays of Objects

Class design and construction

issued belongs to the class and is shared by all tickets. CAPACITY is also a class wide constant. seat and name belong to individual objects. The constructor leaves seat zero for waiting tickets and increments issued only while a seat is available. int BusTicket::issued = 0; is the single namespace scope definition required by this non inline static data member. The array initializer constructs four tickets in order, yielding seats 1, 2, 3 and then Waiting. count() reports three without requiring a particular object.

Static member rules and lifetime

A static member function has no this pointer. It can use static members directly; it can use an instance member only through an explicitly supplied object. A static counter survives the destruction of local ticket arrays. Calling issue() twice constructs four objects: the first call gets seats 1 and 2; the second gets seat 3 and Waiting. The counts printed are 2 then 3. Loop iterations and function calls do not reset class storage. This design counts seats ever allocated for one trip, not currently live tickets; it deliberately does not release seats in a destructor.

Token prediction

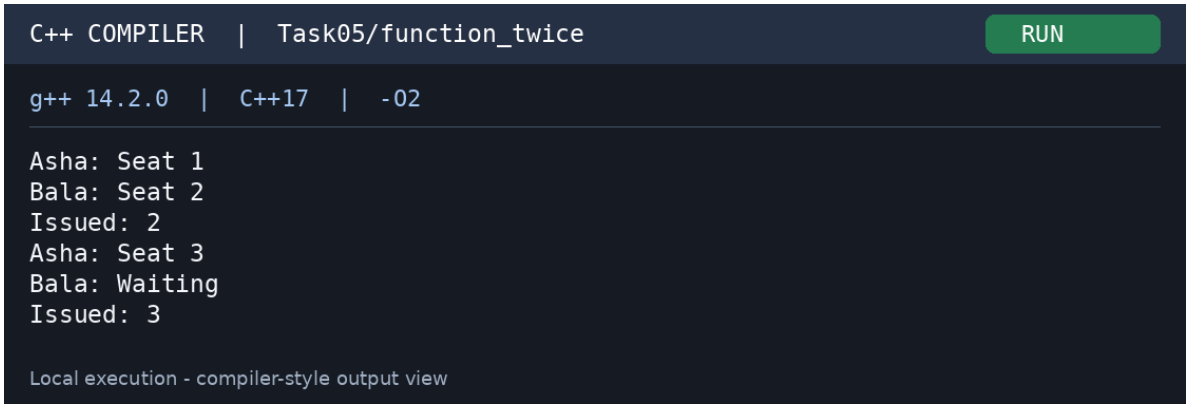
Token::next starts at 100 because of its out of class definition. The three array constructors assign 101, 102 and 103; extra assigns 104. Four constructors ran, so issued() returns 104-100 = 4. The exact line is 101 103 104 4. Execution matched the prediction.

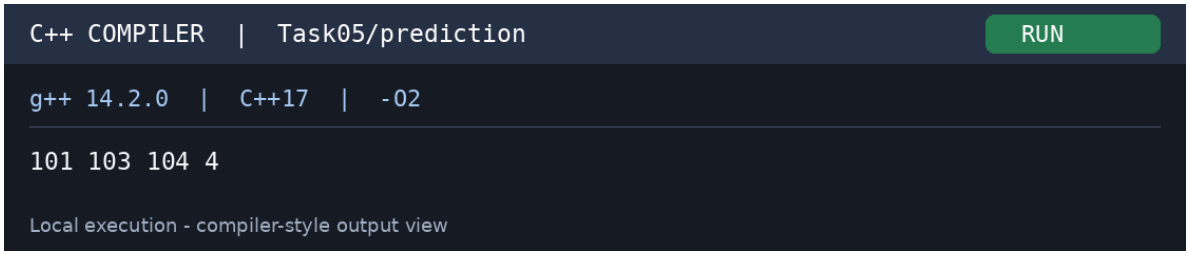
Alternatives and complexity

A global int issued could implement the same O(1) allocation with one shared counter, but exposes mutable state outside the class and mixes unrelated buses. A TicketOffice with capacity and issued members can allocate seats through office.allocate(), while BusTicket receives a TicketOffice& in its constructor. Each bus then owns its own office and counter, permitting two buses at once without reset conflicts. All versions allocate each seat in O(1); an n object array takes O(n) construction time and storage. The static and global versions use one counter overall; the office design uses one counter per independent bus. Copying a ticket would duplicate its seat identity, so a production ticket type should disable copying or explicitly define transfer semantics. Concurrent allocation also requires synchronization.

5) Execution evidence







5) Program source

complete.cpp

```
#include <iostream>
#include <string>
class BusTicket{
 static int issued;static const int CAPACITY=3;int seat;std::string name;
public:
 explicit BusTicket(const std::string& n):seat(0),name(n){if(issued<CAPACITY)seat=++issued;}
 static int count(){return issued;}
 void print()const{std::cout<<name<<": "<<(seat?"Seat
"+std::to_string(seat):std::string("Waiting"))<<'\n';}
};
int BusTicket::issued=0;
int main(){BusTicket
trip[4]={BusTicket("Asha"),BusTicket("Bala"),BusTicket("Chitra"),BusTicket("Dev")};
 for(const BusTicket& t:trip){t.print();}std::cout<<"Issued: "<<BusTicket::count()<<'\n';}
```

function_twice.cpp

```
#include <iostream>
#include <string>
class BusTicket{
 static int issued;static const int CAPACITY=3;int seat;std::string name;
public:
 explicit BusTicket(const std::string& n):seat(0),name(n){if(issued<CAPACITY)seat=++issued;}
 static int count(){return issued;}
 void print()const{std::cout<<name<<": "<<(seat?"Seat
"+std::to_string(seat):std::string("Waiting"))<<'\n';}
};
int BusTicket::issued=0;

void issue(){BusTicket trip[2]={BusTicket("Asha"),BusTicket("Bala")};
 for(const auto& t:trip){t.print();}std::cout<<"Issued: "<<BusTicket::count()<<'\n';}
```

```
int main(){issue();issue();}
```

prediction.cpp

```
#include <iostream>
class Token{static int next;int id;public:Token(){id=++next;}int get()const{return id;}static int
issued(){return next-100;}};
int Token::next=100;
int main(){Token counter[3];Token extra;std::cout<<counter[0].get()<<' '<<counter[2].get()<<'
'<<extra.get()<<' '<<Token::issued()<<'\n';}
```

6) Friend Functions and Inline Functions

Comparison design and correctness

minutesBetween(a,b) treats its two Time operands symmetrically and computes the absolute difference in minutes since midnight. earlier(a,b) compares the same totals. Friendship permits direct access to both objects' private h and m, even though these are free functions called without a receiver object. A member alternative is a.minutesTo(b) and a.isEarlier(b); a Time member may also access another Time object's private state. Friendship is a design choice, not a requirement for comparing two objects: getters are another option.

Hand verification

09:45 is 585 minutes and 13:10 is 790 minutes, so the difference is 205 and earlier(arrival,departure) is false. 11:50 is 710 and 12:10 is 730, giving 20 and true. Comparing 09:45 to itself gives zero and false. All observed values match. These functions compare times within the same day, not overnight journey durations. The supplied completion preserves the given constructor; a deployed constructor should reject hours outside 0..23 and minutes outside 0..59.

Inline investigation and method

inline permits matching definitions in multiple translation units under the language rules; it does not force the optimizer to replace a call. Functions defined inside the class are implicitly inline in this ordinary context. An optimizer can inline a function without that keyword, and can leave an inline function as a call. benchmark.cpp measures ten million comparisons per row at both -O0 and -O2. A volatile varying seed prevents replacing the entire loop with one precomputed answer, and the printed checksum keeps computed results observable. Each row includes object construction, loop and checksum work, so it is an end to end microbenchmark, not an isolated call latency. Three runs per optimization level are retained; the report presents the first run and the median summary.

Deployment decision and alternatives

I retain a small symmetric nonmember comparison interface. Friendship limits access to explicitly declared functions; making h and m public would permit invalid states and tie clients to storage representation. Public getters expose a stable read interface with less privileged free functions; members keep operations with the type but introduce an asymmetric call syntax. All three calculate a fixed number of integer operations in O(1) time and O(1) extra space, and const references avoid

copies of the operands. Differences in these small measurements cannot establish a general speed advantage. The implementation may optimize ordinary and inline functions alike; the measured results below determine the conclusion for this environment.

Measured timing results

Task06/benchmark_O0

regular friend: median 152.6230 ms; runs 152.623, 126.751, 161.351

inline friend: median 128.1670 ms; runs 128.167, 117.91, 134.514

member: median 151.6290 ms; runs 156.915, 129.6, 151.629

getters: median 190.5070 ms; runs 235.589, 129.664, 190.507

Task06/benchmark_O2

regular friend: median 25.2394 ms; runs 32.3814, 25.2394, 22.1688

inline friend: median 24.1986 ms; runs 40.8609, 22.8355, 24.1986

member: median 25.3361 ms; runs 28.0202, 23.1698, 25.3361

getters: median 29.4044 ms; runs 30.2504, 23.7675, 29.4044

At -O0 the ordinary and inline friend medians were 152.6230 and 128.1670 ms; at -O2 they were 25.2394 and 24.1986 ms. These observations do not isolate the inline keyword: optimizer choices, code layout and system load affect the measurements. Equal checksums confirm equal accumulated results.

6) Execution evidence

```
C++ COMPILER  |  Task06/benchmark_O0                                    RUN

g++ 14.2.0  |  C++17  |  -O0

regular friend: 152.623 ms checksum 3634577920
inline friend: 128.167 ms checksum 3634577920
member: 156.915 ms checksum 3634577920
getters: 235.589 ms checksum 3634577920

Local execution - compiler-style output view
```

```
C++ COMPILER  |  Task06/benchmark_O2                                    RUN

g++ 14.2.0  |  C++17  |  -O2

regular friend: 32.3814 ms checksum 3634577920
inline friend: 40.8609 ms checksum 3634577920
member: 28.0202 ms checksum 3634577920
getters: 30.2504 ms checksum 3634577920

Local execution - compiler-style output view
```

```
C++ COMPILER  |  Task06/complete                                        RUN

g++ 14.2.0  |  C++17  |  -O2

205 false

Local execution - compiler-style output view
```

```
C++ COMPILER  |  Task06/correctness                                     RUN

g++ 14.2.0  |  C++17  |  -O2

Across noon: 20 true
Same time: 0 false

Local execution - compiler-style output view
```

correctness execution evidence

6) Program source

benchmark.cpp

```
#include <chrono>
#include <iostream>
class Time{
 int h,m;
public:
 Time(int hh,int mm):h(hh),m(mm){}
 int total()const{return h*60+m;}
 int minutesTo(const Time& b)const{int d=total()-b.total();return d<0?-d:d;}
 friend int regular(const Time& a,const Time& b);
 friend int inlined(const Time& a,const Time& b);
};
int regular(const Time& a,const Time& b){int d=(a.h*60+a.m)-(b.h*60+b.m);return d<0?-d:d;}
inline int inlined(const Time& a,const Time& b){int d=(a.h*60+a.m)-(b.h*60+b.m);return d<0?-d:d;}
int getters(const Time& a,const Time& b){int d=a.total()-b.total();return d<0?-d:d;}
template<class F>void measure(const char* name,F f){volatile int seed=7;long long sum=0;
 auto start=std::chrono::steady_clock::now();for(int i=0;i<10000000;++i){int x=(i+seed)%1440;Time
a(x/60,x%60),b(13,10);sum+=f(a,b);}
 auto end=std::chrono::steady_clock::now();std::cout<<name<<":
"<<std::chrono::duration<double,std::milli>(end-start).count()<<" ms checksum "<<sum<<'\n';}
int main(){measure("regular friend",[](const Time&a,const Time&b){return regular(a,b);});
 measure("inline friend",[](const Time&a,const Time&b){return inlined(a,b);});
 measure("member",[](const Time&a,const Time&b){return a.minutesTo(b);});
 measure("getters",[](const Time&a,const Time&b){return getters(a,b);});}
```

complete.cpp

```
#include <iostream>
class Time{int h,m;public:Time(int hh,int mm):h(hh),m(mm){}
 friend int minutesBetween(const Time& a,const Time& b);
 friend bool earlier(const Time& a,const Time& b);
};
```

```
int minutesBetween(const Time& a,const Time& b){int d=(a.h*60+a.m)-(b.h*60+b.m);return d<0?-
d:d;}
bool earlier(const Time& a,const Time& b){return a.h*60+a.m<b.h*60+b.m;}
int main(){Time departure(9,45),arrival(13,10);
 std::cout<<minutesBetween(departure,arrival)<<' '<<std::boolalpha<<earlier(arrival,departure)<<'\n';}
```

correctness.cpp

```
#include <iostream>
class Time{int h,m;public:Time(int hh,int mm):h(hh),m(mm){}
 friend int minutesBetween(const Time& a,const Time& b);
 friend bool earlier(const Time& a,const Time& b);
};
int minutesBetween(const Time& a,const Time& b){int d=(a.h*60+a.m)-(b.h*60+b.m);return d<0?-
d:d;}
bool earlier(const Time& a,const Time& b){return a.h*60+a.m<b.h*60+b.m;}

int main(){Time a(11,50),b(12,10),c(9,45);
 std::cout<<std::boolalpha<<"Across noon: "<<minutesBetween(a,b)<<' '<<earlier(a,b)<<'\n';
 std::cout<<"Same time: "<<minutesBetween(c,c)<<' '<<earlier(c,c)<<'\n';}
```

7) Diagnose Redesign and Validate

Diagnose the limitation

Q4 shifts K-1 old transactions on every insertion after the history fills. The requirement change from ten to fifty increases repeated copying; scaling further makes insertion work grow with capacity. This follows directly from the record loop, and the paired benchmark measures element writes and elapsed time. I isolate the same integer history policy in both versions to avoid mixing account formatting or balance calculations into the comparison.

Define a measurable objective

For 200000 insertions, retain exactly the same last K values in chronological order at K=10, 50 and 1000. At K=1000, reduce data element writes by at least 99% and make insertion independent of K. Compare elapsed time using the same compiler, input sequence and checksum visible results. Timing improvement at the smallest capacity is desirable but is not a correctness condition.

Revised design and trade off

History owns a vector allocated once at construction, count, start and a policy flag selecting shifting or a circular array. The vector provides capacity storage only; no deque or library rotation algorithm replaces the design. Both paths use the same underlying allocation. Once full, the circular path overwrites the oldest slot and advances start modulo K. Traversal starts at start and wraps. It uses O(K) storage and one extra O(1) index, but chronological storage is no longer contiguous from element zero. The test assumes K>0. In a production class, reject a zero capacity and use a template or a dedicated circular class to remove the policy flag.

Circular insertion pseudocode

If count < capacity: data[count] = value; count = count + 1. Otherwise: data[start] = value; start = (start + 1) modulo capacity. To display: for i from 0 to count-1, read data[(start+i) modulo capacity]. The invariant is that start identifies the oldest retained entry once full. A capacity of fifty changes only construction, not insertion or traversal logic.

Prediction before execution

The circular version should perform exactly 200000 element writes at every capacity. The shifting version should perform K + (200000-K)*K writes. Both should retain [200001-K, 200000]. For eleven

values at capacity ten, both should retain 2 through 11. The ring should show greater benefit as K grows, although cache effects and vectorized shifts may let shifting compete at K=10.

Implementation and validation

redesign.cpp runs both versions on the same sequence, compares the full retained vectors, reports each write count and timing, and returns failure if they differ. Three -O2 runs are retained. The first run is shown in the execution image; the median summary below uses all three. Ring insertion is O(1); full shifting insertion is O(K). Producing either retained sequence is O(K). The benchmark allocates outside the timed insertion loop and includes policy checks and write counting in both timed paths.

Defend the final design

All equality tests passed, including wraparound at capacity ten. At capacity 1000 the ring performs 200000 writes compared with 199001000 for shifting, a 99.8995% reduction, exceeding the objective. The capacity independent write count is deterministic evidence even when timing varies. The measured timings below support a decision for this compiler and machine, not a universal speed guarantee. I choose the circular design for growing histories. The original shift design remains easy to inspect and can be adequate for ten entries when insertion is rare. Extra index state and wrapped traversal are the main costs of the redesign.

Median timing comparison

k=10 shift: median 2.0411 ms; runs 1.54766, 2.46404, 2.04107

k=10 ring: median 0.8434 ms; runs 0.721858, 0.843439, 1.05092

k=50 shift: median 3.5412 ms; runs 17.8163, 3.36109, 3.54118

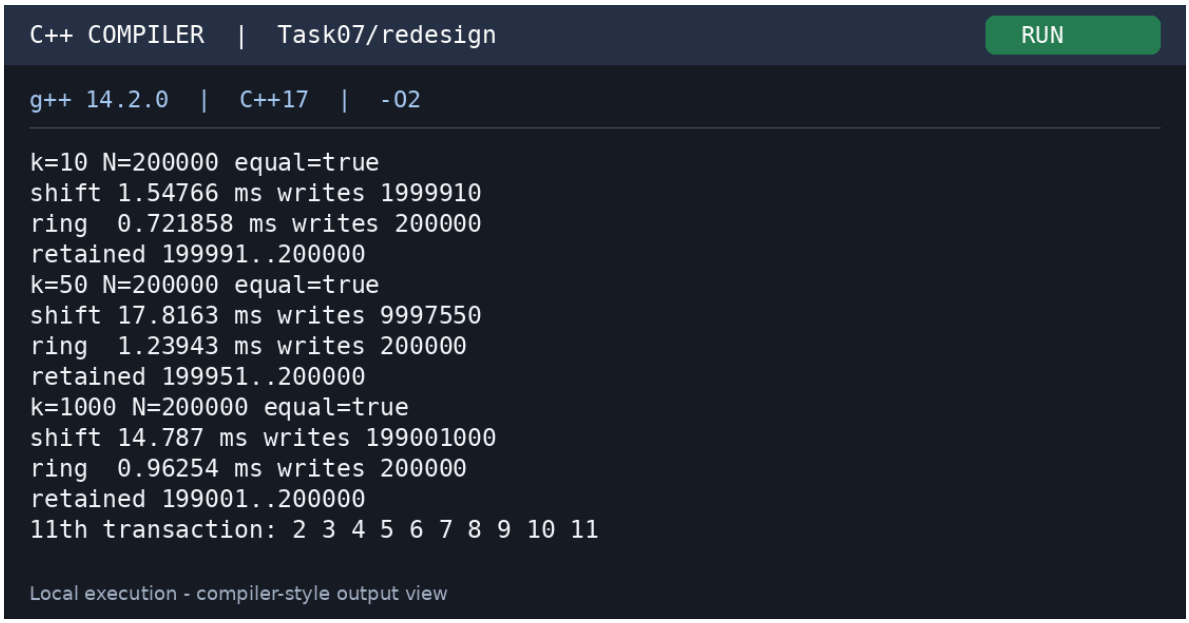
k=50 ring: median 0.9927 ms; runs 1.23943, 0.702966, 0.992709

k=1000 shift: median 16.4442 ms; runs 14.787, 19.1222, 16.4442

k=1000 ring: median 0.8829 ms; runs 0.96254, 0.675464, 0.882873

The timings include instrumentation and are single process microbenchmarks. The deterministic write count and equality checks provide stronger evidence of asymptotic behaviour than one elapsed time result.

7) Execution evidence



7) Program source

redesign.cpp

```
#include <algorithm>
#include <chrono>
#include <iostream>
#include <vector>
class History{
 std::vector<int> a;int count=0,start=0;bool ring;
public:
 long long moves=0;
 History(int k,bool r):a(k),ring(r){}
 void add(int v){int k=static_cast<int>(a.size());
 if(count<k){a[count++]=v;++moves;}
 else if(ring){a[start]=v;start=(start+1)%k;++moves;}
 else{for(int i=1;i<k;++i){a[i-1]=a[i];++moves;}a[k-1]=v;++moves;}}
 std::vector<int> values()const{std::vector<int> v;for(int
i=0;i<count;++i)v.push_back(a[(start+i)%a.size()]);return v;}
};
int main(){constexpr int N=200000;
 for(int k:{10,50,1000}){History shift(k,false),ring(k,true);
 auto run=[&](History& h){auto t=std::chrono::steady_clock::now();for(int i=1;i<=N;++i)h.add(i);
 return std::chrono::duration<double,std::milli>(std::chrono::steady_clock::now()-t).count();};
 double s=run(shift),r=run(ring);auto a=shift.values(),b=ring.values();
 std::cout<<"k="<<k<<" N="<<N<<" equal="<<std::boolalpha<<(a==b)<<'\n';
 std::cout<<"shift "<<s<<" ms writes "<<shift.moves<<"\nring  "<<r<<" ms writes
"<<ring.moves<<'\n';
 std::cout<<"retained "<<b.front()<<".."<<b.back()<<'\n';if(a!=b)return 1;}
 History h(10,true);for(int i=1;i<=11;++i)h.add(i);std::cout<<"11th transaction:";for(int
v:h.values())std::cout<<' '<<v;std::cout<<'\n';}
```